Bachelor en Sciences Informatiques

Systèmes Informatiques – 8.io_2.pdf

1. *Que font les programmes suivant ?*

   Les programmes suivant simule un compte en banque sur lequel on peut retirer de l'argent s'il le montant demandé est égal ou plus petit que le montant sur le compte.

2. *Décrivez en détail le code et son fonctionnement.*

initialize.c

```c
#include <unistd.h>
#include <stdlib.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>


int main( int argc, char **argv ) {
  char* dataFile = argv[1];
  int fd = open( dataFile, O_WRONLY|O_CREAT|O_TRUNC, 0600 );
  int accounts[] = { 100, 20, 50, 1000, 5 };
  write( fd, accounts, sizeof(int) * 5 );
  close(fd);
  return EXIT_SUCCESS;
}
```

Ce programme prend en paramètre l'entrée de l'utilisateur *argv[1]* – un fichier - avec lequel il initialise *dataFile*.
Il ouvre ensuite ce fichier avec *open()* en écriture seulement et la longueur du fichier sera tronqué à 0 si c'est un fichier régulier. Si ce fichier est un FIFO. Alors le drapeau O_TRUNC est ignoré. Si ce fichier n'existe pas, alors il est créer avec la permission 0600 = user write and read.
open() retourne un descripteur de fichier (positif integer) dans *fd* et retourne -1 s'il y a eu une erreur.
Un tableau d'integer est initialisé avec 5 éléments.
write() écrit dans le fichier les éléments qu'il y a dans le tableau.
close() ferme le fichier

operations.c

```c
#include <unistd.h>
#include <stdlib.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <stdio.h>

#define ACC_SIZE 5
#define SIZE sizeof(int)

void display( int fd ) {
  int i,a;
  lseek( fd, 0, SEEK_SET );
  for( i=0; i < ACC_SIZE; i++ ) {
    read( fd, &a, SIZE );
    printf( "%d: %d\n", i, a );
  }
}

int get( int fd, int acc ) {
  int off = acc * SIZE;
  int a = -1;
  lseek( fd, off, SEEK_SET );
  read( fd, &a, SIZE );
  return a;
}
```

Soit la fonction *display()* qui ne retourne rien et qui prend en paramètre un descripteur de fichier.
lseek() place la tête de lecture/écriture en partant du début du fichier (SEEK_SET) à la position de l'offset, ici 0.
Boucle *for* qui va lire le descripteur de fichier et ensuite afficher les variables *i* et *a*.

Soit la fonction *get()* qui prend en paramètre un descripteur de fichier et un integer.
lseek() place la tête de lecture/écriture en partant du début du fichier (SEEK_SET) à la position de l'offset, ici *off*.
read() va lire le fichier jusqu'à SIZE octets dans le tampon pointé par a.

```
27
28 void set( int fd, int acc, int total ) {
29   int off = acc * SIZE;
30   lseek( fd, off, SEEK_SET );
31   write( fd, &total, SIZE );
32 }
33
34 void acquire( int fd, int acc ) {
35   struct flock fl;
36   fl.l_type = F_WRLCK;
37   fl.l_whence = SEEK_SET;
38   fl.l_start = acc;
39   fl.l_len = 1;
40   fcntl(fd, F_SETLKW, &fl);
41 }
42
43 void transfer( int fd, int from, int to, int amount ) {
44   int fromTotal, toTotal;
45   acquire( fd, from );
46   fromTotal = get( fd, from );
47   if( fromTotal < amount ) {
48     printf("Not enough money in account: %d\n", from );
49     return;
50   }
51   acquire( fd, to );
52   toTotal = get( fd, to );
53   set( fd, from, fromTotal-amount );
54   set( fd, to, toTotal+amount );
55 }
56
```

Soit la fonction *set()* qui prend en paramètre un descripteur de fichier, et deux integers.
lseek() place la tête de lecture/écriture en partant du début du fichier (SEEK_SET) à la position de l'offset, ici off.
write() écrit jusqu'à SIZE dans le descripteur de fichier depuis le tampon pointé par &total.

Soit la fonction *acquire()* qui prend un descripteur de fichier et un int.
Une structure *flock* y est définit, ce qui permet de poser un verrou sur un fichier. Le verrou est seulement en écriture (F_WRLCK). Le départ est par le début du fichier, le starting offset vaut *acc* et la longueur du verrou est défini par *fl-l_len* (Note : si *fl.l_len* vaut 0, alors le verrou va jusqu'à la fin du fichier).
fcntl() est la fonction qui permet de poser ce verrou précédemment définit.

Soit la fonction *transfer()* qui prend en paramètre un descripteurs de fichier et 3 int.
Elle appelle la fonction *acquire()* pour poser un verrou. Ensuite elle va lire une partie du fichier et récupérer ses données avec *get()* et les comparer avec *amount*.

3. *Ce code peut poser problème si plusieurs processus indépendants essayent d'accéder simultanément au même compte. Expliquez ?*

   Supposons deux processus A et B qui accèdent au compte, si A pose un verrou pour retirer un certaine somme et que B accède au compte avant de que la somme soit débitée, il obtiendra une information fausse sur la quantité d'argent sur le compte et pourra retirer un montant trop grand par rapport à la réelle somme qu'il y a dessus.

4. *Comment résoudre ce problème ?*

   On peut résoudre ce problème en posant un verrou readlock.